

DAT32-4

APIs, Automation & Data Pipelines

Building reproducible data workflows

A pipeline must re-run without duplicating data or losing records.

Albert School — 16 hours

Contents

1 REST APIs and the request-response model	2
2 API authentication	2
3 Pagination	3
4 ETL pipeline design	3
5 Scheduling	3
6 Logging	3
7 Pipeline failure modes	4
8 Idempotency	4
9 Pipeline documentation	4

1 REST APIs and the request-response model

A REST API is an interface that lets a program request data from a server over HTTP. Communication follows the **request-response model**: the client sends a request to a URL (an endpoint), and the server returns a response, typically with a body in JSON. This builds on the basic HTTP GET requests and JSON parsing with the `requests` library introduced in DAT12-2.

Definition 1 A **REST API** exposes resources at URLs (endpoints). A client sends an HTTP **request** specifying a method (GET, POST, ...) and an endpoint; the server returns a **response** containing a status code and, usually, a body.

Every response carries an **HTTP status code** indicating the outcome. Codes are grouped by their first digit:

- **2xx** — success (e.g. 200 OK).
- **3xx** — redirection.
- **4xx** — client error: the request was wrong (e.g. 401 Unauthorized, 404 Not Found, 429 Too Many Requests).
- **5xx** — server error: the server failed (e.g. 500 Internal Server Error, 503 Service Unavailable).

Interpreting the status code is the first step in deciding whether a request succeeded, must be corrected, or should be retried.

2 API authentication

Most APIs require the client to prove its identity before returning data. Two mechanisms are used.

Definition 2 An **API key** is a single secret string sent with each request (often as a header or query parameter) that identifies the caller.

Definition 3 An **OAuth token** is a credential obtained through an authorization flow. The client exchanges its credentials for a token, which is then sent with each request and which typically expires and must be refreshed.

The security difference: an API key is a long-lived static secret — if it leaks, the holder has full access until the key is revoked. An OAuth token is short-lived and scoped, limiting the damage of a leak and allowing access to be granted without sharing a permanent secret. In both cases the

credential must be kept out of source code (e.g. in an environment variable), never committed to version control.

3 Pagination

An endpoint that returns many records does not return them all at once. It returns one **page** of results, plus a way to request the next page. Collecting the full result set requires iterating over pages until none remain.

Definition 4 Pagination is the division of a large result set into successive pages. The client requests pages in turn — following a **next** link, incrementing a page number, or passing a cursor — until the API signals that no more results remain.

The collected results are then stored in a structured format (e.g. a list of records, written to a CSV or a database table) as covered in DAT12-2 and DAT22-5.

Common pitfall Stopping after the first page yields an incomplete dataset. The loop must continue until the API's own end-of-results signal, not until a fixed number of pages.

4 ETL pipeline design

A data pipeline is organised as three clearly separated steps.

Definition 5 ETL stands for **Extract, Transform, Load**:

- **Extract** — read data from the source (here, an API).
- **Transform** — clean, reshape, and validate the data.
- **Load** — write the result into a destination (here, a database).

Keeping the three steps separated — typically as distinct functions — means each can be tested, debugged, and re-run independently. Extraction code does not clean data; transformation code does not call the API or the database; loading code does not reshape data. This separation reuses the function and object-oriented design discipline from DAT12-2 and DAT22-3.

5 Scheduling

A pipeline that must run regularly is executed automatically rather than by hand.

Definition 6 Scheduling is the automatic execution of a script at defined times. On Linux and macOS this is done with **cron**; on Windows with **Task Scheduler**.

A cron schedule is defined by a **crontab** entry with five time fields — minute, hour, day of month, month, day of week — followed by the command to run. For example, an entry running a script every day at 02:00 specifies minute 0, hour 2, and * for the remaining fields. Task Scheduler expresses the same idea through its own trigger configuration.

6 Logging

When a scheduled script runs unattended, there is no one watching its output. To diagnose a failure afterwards, the script must record what it did.

Definition 7 Logging is the recording of timestamped messages describing a program's execution — what step it reached, what it processed, and what errors occurred — to a file or stream that can be inspected later.

Python's `logging` module is used instead of `print`: it attaches a timestamp and a severity level (`INFO`, `WARNING`, `ERROR`) to each message and can write to a file. The goal is that a failure can be diagnosed from the log alone, without re-running the code.

7 Pipeline failure modes

A pipeline reading from an external API fails in predictable ways. Three are the most common and must be handled explicitly.

Definition 8 The three common **failure modes** are:

- **Source unavailability** — the API does not respond or returns a 5xx error.
- **Schema changes** — the API's response structure changes (a field is renamed, removed, or retyped), breaking the transformation step.
- **Duplicate records** — the same records are fetched and loaded more than once, e.g. on a re-run.

Source unavailability is handled by detecting the error (via the status code) and retrying or failing cleanly with a logged message rather than crashing silently. Schema changes are handled by validating the response structure before transforming, and logging a clear error when an expected field is absent. Duplicate records are addressed by idempotency.

8 Idempotency

Definition 9 A pipeline is **idempotent** when re-running it on the same input data produces the same result — no duplicated records and no lost records — regardless of how many times it is run.

Idempotency is what makes a pipeline safe to re-run after a failure. It is typically achieved by giving each record a unique key and loading with an **upsert** — insert a record if its key is absent, update it if present — rather than a blind insert. The SQL skills from DAT22-5 (keys, `INSERT`, `UPDATE`, deduplication) are the tools used here.

Common pitfall A blind `INSERT` on every run is not idempotent: re-running the pipeline appends the same records again, duplicating the data. Re-running must be a no-op when the data has not changed.

9 Pipeline documentation

A pipeline must be understandable by a colleague without reading its code.

Definition 10 Pipeline documentation states the pipeline's **inputs** (its source and required credentials), its **outputs** (the destination and the resulting structure), and its **failure behavior** (what happens on source unavailability, schema changes, and re-runs).

This documentation — typically a README, building on the project-structure discipline from DAT22-3 — is the contract for the pipeline: it lets a colleague operate, schedule, and troubleshoot the pipeline from the description alone.

Exercises

1. Explain what a REST API is and describe the request-response model. Given a list of HTTP status codes, state for each whether it indicates success, a client error, or a server error.
2. Authenticate against an API once using an API key and once using an OAuth token. Explain the security difference between the two.
3. Paginate through a paginated API endpoint, collect all results, and store them in a structured format.
4. Design a Python pipeline with extraction, transformation, and loading as clearly separated steps.
5. Schedule a Python script to run automatically every day using cron (Linux/ macOS) or Task Scheduler (Windows).
6. Add logging to a script so that a failure can be diagnosed from the log file without re-running the code.
7. Make a pipeline handle the three common failure modes: source API unavailability, a schema change in the response, and duplicate records.
8. Make a pipeline idempotent so that re-running it on the same data produces no duplicates and loses no records. Verify by running it twice.
9. Write documentation for a pipeline stating its inputs, outputs, and failure behavior, such that a colleague can understand it without reading the code.